

Informe de QA - Perfum Lab System

Proyecto: Perfum Lab System

Fecha: 3 de agosto de 2026

Tipo de documento: README tecnico de pruebas QA

Modulo evaluado: Productos, inventario, ventas, facturas y reportes

Resumen ejecutivo

Se implemento una base de pruebas automatizadas para validar los flujos principales del sistema Perfum Lab sin modificar los datos reales del negocio.

El trabajo de QA se centro en verificar que las operaciones criticas funcionen correctamente:

- Gestion de productos.
- Movimientos de inventario.
- Registro de ventas simples y multiples.
- Validaciones de stock.
- Anulacion de ventas.
- Generacion de facturas.
- Exportacion de facturas en TXT y PDF.
- Reportes operativos.

Las pruebas se ejecutan sobre una base JSON temporal creada durante cada corrida. Esto permite probar el sistema de forma segura, repetible y sin contaminar los archivos reales ubicados en database/json/.

Objetivo del QA

El objetivo fue crear una forma confiable de validar automaticamente que el sistema mantenga la integridad de los datos cuando se realizan operaciones comerciales importantes.

En particular, se busco comprobar que:

- Una venta descuenta correctamente el inventario.
- Una anulacion devuelva correctamente el stock.
- Una venta facturada no pueda ser anulada.
- Una venta anulada no pueda ser facturada.
- Las ventas invalidas no creen registros ni modifiquen inventario.
- Las facturas puedan consultarse y exportarse.
- Los reportes reflejen correctamente ventas, facturas y productos con bajo stock.

Archivos trabajados

Archivos modificados

Archivo | Proposito

'app/ventas/ventas.py' | Se agrego soporte para ejecutar funciones contra una ruta de datos temporal mediante 'ruta_db'.

'app/facturas/facturas.py' | Se agrego soporte para generar, consultar y exportar facturas usando una base JSON temporal.

'app/reportes/reportes.py' | Se agrego soporte para consultar reportes desde una ruta JSON temporal.

'probar_ventas_facturas.py' | Se completo como prueba rapida tipo smoke test para ventas, facturas, PDF y reportes.

Archivos agregados

Archivo | Proposito

'tests/test_qa_core.py' | Suite automatizada de QA con pruebas unitarias e integradas usando 'unittest'.

'docs/README_QA.md' | Documento de QA listo para exportar a PDF.

Enfoque tecnico

Antes del QA, varias funciones de ventas, facturas y reportes usaban directamente la base JSON real del proyecto.

Para probar sin riesgo, se agrego el parametro opcional:

```
ruta_db=DATABASE_PATH
```

Esto permite dos modos de uso:

Modo normal de la aplicacion

La aplicacion sigue funcionando igual, usando los datos reales:

```
ventas.registrar_venta(1, "Cliente", 2)
```

Modo QA con datos temporales

Las pruebas pueden usar una carpeta temporal:

```
ventas.registrar_venta(1, "Cliente QA", 2, ruta_db=ruta_temporal)
```

De esta forma, cada prueba crea sus propios archivos JSON, ejecuta el flujo necesario y elimina esos datos al finalizar.

Pruebas automatizadas implementadas

La suite principal se encuentra en:

```
tests/test_qa_core.py
```

Se implementaron 7 pruebas automatizadas.

5.1 Productos e inventario

Prueba:

```
test_productos_e_inventario_crud_y_movimientos
```

Valida:

- Crear producto.
- Rechazar SKU duplicado.
- Buscar producto por texto.
- Registrar entrada de inventario.
- Registrar salida de inventario.
- Rechazar salida mayor al stock disponible.
- Registrar ajuste de inventario.
- Consultar movimientos.
- Desactivar producto.

Resultado esperado:

- El stock cambia correctamente en cada movimiento.
- Los movimientos quedan registrados con tipo, cantidad, stock anterior y stock nuevo.
- El producto desactivado ya no aparece en el listado de activos.

5.2 Venta multiple, factura y exportaciones

Prueba:

```
test_venta_multiple_factura_exports_y_bloqueo_de_anulacion
```

Valida:

- Registrar venta con mas de un producto.
- Calcular cantidad total de la venta.
- Calcular total monetario.
- Descontar inventario de cada producto vendido.
- Consultar detalle de venta.
- Generar factura.
- Evitar factura duplicada.
- Evitar anulacion de venta facturada.
- Generar texto de factura.
- Exportar factura TXT.
- Exportar factura PDF.

Resultado esperado:

- La venta queda en estado Completada.
- La factura queda con numero FAC-000001.
- El PDF generado inicia con cabecera valida %PDF-.
- Una venta facturada no puede anularse.

5.3 Anulacion de venta

Prueba:

test_anular_venta_devuelve_stock_y_excluye_facturacion

Valida:

- Registrar una venta.
- Descontar stock.
- Anular la venta.
- Devolver stock al producto.
- Cambiar estado de la venta a Anulada.
- Evitar facturar una venta anulada.

Resultado esperado:

- El stock vuelve al valor inicial.
- La venta queda anulada.
- El sistema rechaza la facturacion de ventas anuladas.

5.4 Validaciones de ventas invalidas

Prueba:

test_validaciones_de_venta_no_modifican_datos

Valida:

- Rechazar venta sin cliente.
- Rechazar venta con cantidad cero.
- Rechazar venta con cantidad mayor al stock disponible.
- Confirmar que no se creen ventas invalidas.
- Confirmar que no se creen movimientos de inventario invalidos.
- Confirmar que el stock no cambie.

Resultado esperado:

- Todas las ventas invalidas fallan.
- No se alteran ventas, movimientos ni inventario.

5.5 Validaciones de formato y campos requeridos

Prueba:

test_validaciones_de_formato_y_campos_requeridos

Valida:

- Rechazar SKU demasiado corto.
- Rechazar SKU con espacios.
- Rechazar producto sin nombre valido.
- Rechazar precios no finitos.
- Rechazar categoria sin nombre valido.
- Rechazar ventas con ID de producto invalido.
- Rechazar ventas con cliente sin letras.
- Rechazar cantidades decimales en ventas.
- Rechazar facturas con ID de venta invalido.
- Rechazar fechas con formato incorrecto.
- Rechazar rangos donde la fecha inicial sea mayor que la fecha final.
- Rechazar exportaciones con extension incorrecta.

Resultado esperado:

- Los datos incompletos o con formato incorrecto no ingresan al sistema.
- Los mensajes de error indican el campo que debe corregirse.
- Las exportaciones respetan el tipo de archivo esperado.

5.6 Agrupacion de productos repetidos

Prueba:

test_items_repetidos_se_agrupan_en_una_sola_linea

Valida:

- Registrar venta con el mismo producto repetido en el carrito.
- Agrupar cantidades en una sola linea de detalle.
- Calcular subtotal correcto.
- Descontar stock segun la suma total.

Resultado esperado:

- El detalle de venta contiene una sola linea.
- La cantidad corresponde a la suma de los items repetidos.

5.7 Reportes

Prueba:

test_reportes_resumen_productos_mas_vendidos_y_csv

Valida:

- Resumen de ventas completadas.
- Total vendido.
- Cantidad de facturas.
- Total facturado.
- Productos con bajo stock.
- Productos mas vendidos.
- Ventas recientes.
- Filtro por rango de fechas.
- Exportacion CSV.

Resultado esperado:

- Solo las ventas completadas cuentan para el resumen principal.
- Las ventas anuladas aparecen en ventas recientes, pero no suman al total vendido.
- El CSV contiene encabezados y datos esperados.

Smoke test

Tambien se completo el archivo:

`probar_ventas_facturas.py`

Este script ejecuta una prueba rapida de extremo a extremo:

1. Crea una base JSON temporal.
2. Registra una venta multiple.
3. Genera una factura.
4. Exporta la factura PDF.
5. Consulta resumen de reportes.
6. Imprime resultado final.

Comando:

```
uv run python probar_ventas_facturas.py
```

Salida esperada:

```
QA smoke OK
Venta generada: #1 | Total: L 2480.00
Factura generada: FAC-000001
```

Como ejecutar las pruebas

Desde la raiz del proyecto:

```
cd C:\Users\yesid\OneDrive\Escritorio\perfumelabsystem\perfumlab-system
```

Ejecutar prueba rapida

```
uv run python probar_ventas_facturas.py
```

Ejecutar suite completa

```
uv run python -m unittest discover -s tests
```

Resultado obtenido

La suite completa fue ejecutada correctamente.

Resultado:

```
.....  
-----  
Ran 7 tests in 1.295s  
  
OK
```

Interpretacion:

- 7 tests ejecutados.
- 0 errores.
- 0 fallos.
- Los flujos principales evaluados pasaron correctamente.

Matriz de cobertura

Area	Cubierto	Evidencia
Productos	Si	Crear, buscar, duplicado SKU, desactivar.
Inventario	Si	Entrada, salida, ajuste, stock insuficiente.
Ventas	Si	Venta simple, venta multiple, items repetidos.
Validaciones de venta	Si	Cliente vacio, cantidad cero, stock insuficiente.
Validaciones de formato	Si	SKU, cliente, IDs, fechas, numeros y extensiones.
Anulaciones	Si	Devolucion de stock y estado 'Anulada'.
Facturas	Si	Generacion, consulta y bloqueo de duplicados.
Exportacion TXT	Si	Archivo generado con datos del cliente.
Exportacion PDF	Si	Archivo generado con cabecera '%PDF-'.
Reportes	Si	Resumen, bajo stock, mas vendidos, ventas recientes.
Exportacion CSV	Si	Archivo CSV con encabezados y datos.
Interfaz grafica Tkinter	Parcial	No se automatizaron clics visuales en ventanas.

Riesgos y pendientes

Aunque las pruebas cubren la logica principal, quedan pendientes recomendados para una fase posterior:

- Automatizar pruebas visuales de la interfaz Tkinter.
- Probar flujos manuales con usuario final: abrir modulos, llenar formularios y validar mensajes.
- Agregar pruebas para archivos JSON corruptos o con estructura invalida.
- Validar comportamiento con grandes volumenes de productos y ventas.
- Agregar pruebas de regresion cuando se modifiquen precios, categorias o estructura de factura.

Guia rapida para exportar este Markdown a PDF

Este archivo puede exportarse a PDF de varias formas.

Opcion 1: Visual Studio Code

1. Abrir docs/README_QA.md.
2. Instalar una extension como Markdown PDF.
3. Ejecutar la opcion Markdown PDF: Export (pdf).

Opcion 2: Pandoc

Si Pandoc esta instalado:

```
pandoc docs\README_QA.md -o docs\README_QA.pdf
```

Opcion 3: Navegador

1. Abrir el Markdown en un visor que renderice Markdown.
2. Usar Imprimir.
3. Seleccionar Guardar como PDF.

Conclusion

El sistema Perfum Lab cuenta ahora con una base de QA automatizada para validar los procesos mas importantes del negocio.

Las pruebas ejecutadas confirman que los flujos principales de productos, inventario, ventas, facturas y reportes funcionan correctamente bajo los escenarios cubiertos.

El enfoque con base JSON temporal permite repetir las pruebas de forma segura en cualquier computadora sin afectar los datos reales del sistema.